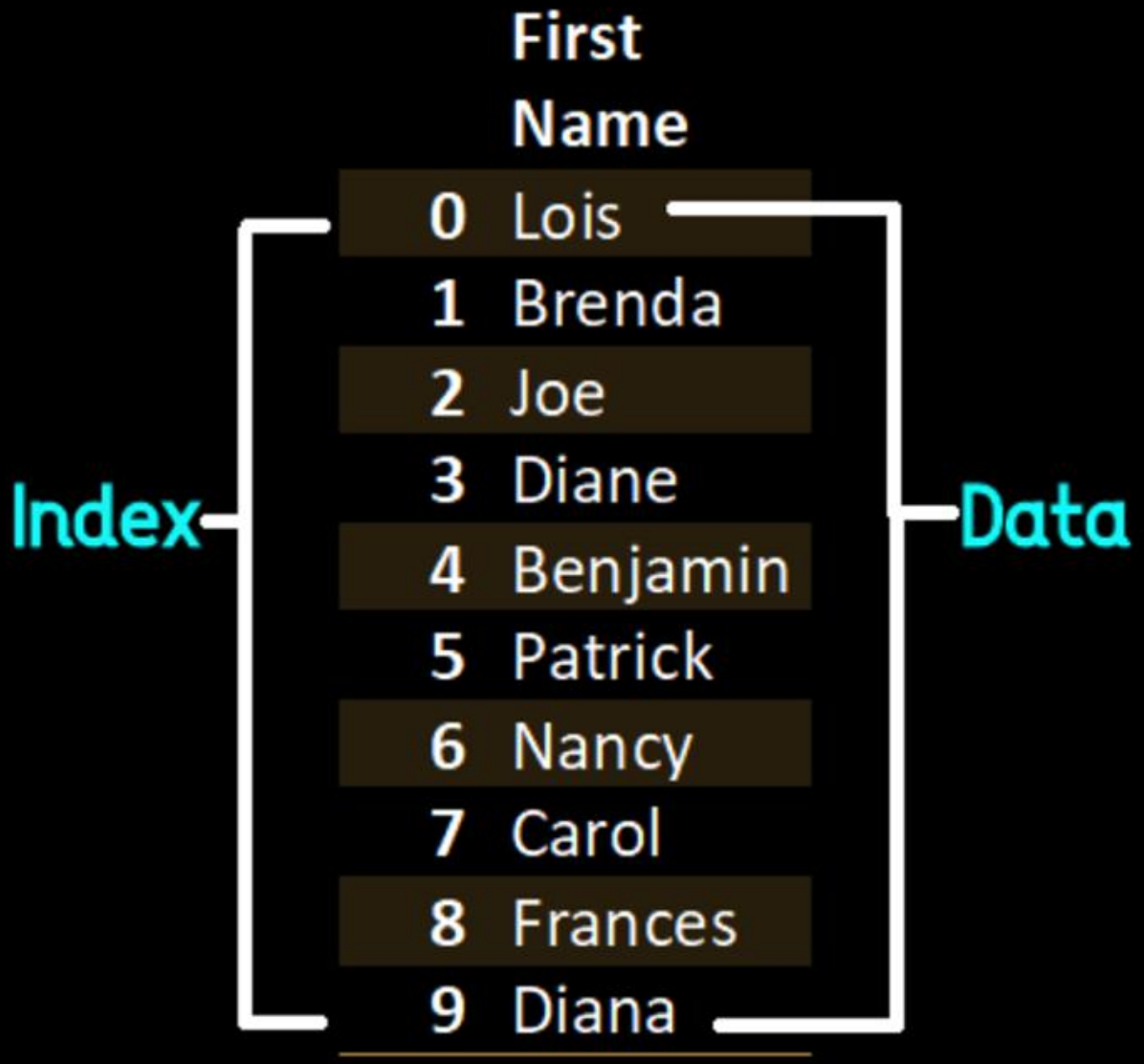


PANDAS SERIES

Series


First
Name
0 Lois
1 Brenda
2 Joe
3 Diane
4 Benjamin
5 Patrick
6 Nancy
7 Carol
8 Frances
9 Diana
Index
Data

What a Series Is

A Series is a 1-D, homogeneously-typed labeled array: a NumPy values buffer plus an Index and a name, exposed through pandas' ExtensionArray API.

-Label lookup s["b"] is O(1) via a hash table built lazily on first access; positional access s.iloc[1] stays O(1) on the underlying buffer.

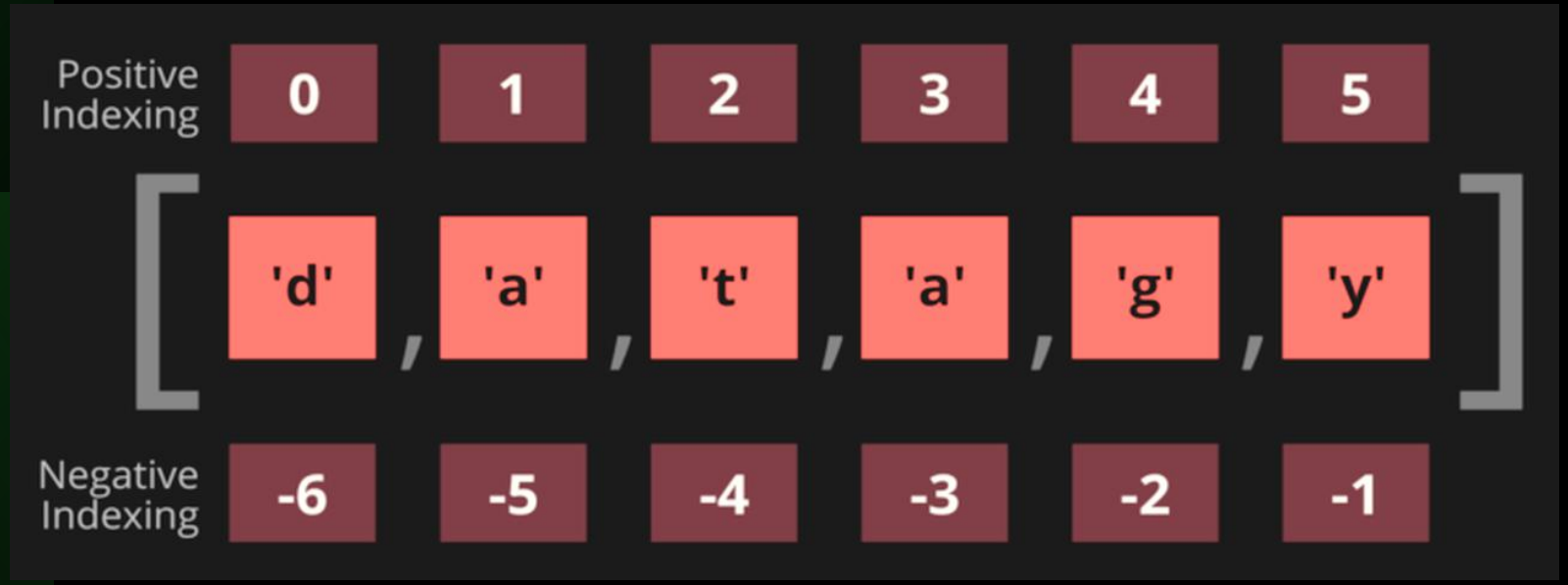
Advantage: one type replaces list, dict and ndarray — dict-like labeled access with array-like vectorized math, and it is exactly a DataFrame column.

Index Mechanics

Index objects are immutable and hashable, so they can be shared across Series and DataFrames with zero copying; slices return views where the dtype allows.

Specialized index types (RangeIndex, DatetimeIndex, CategoricalIndex, MultiIndex) carry their own O(1)/O(log n) lookup engines and memory-efficient storage.

Advantage: automatic label alignment on every binary op joins by label, fills non-matching labels with NaN, and eliminates whole classes of off-by-one merge bugs.



Data Types & dtype Inference

dtype is inferred once at construction and applies to the whole array: int64/float64, bool, datetime64[ns], timedelta64[ns], category, or the nullable Int64/boolean/string extension dtypes.

Mixed or unconvertible values fall back to object dtype, which stores Python pointers and forfeits vectorization — cast explicitly with .astype() to keep operations in C.

Type	Constructor parameter
Data	ndarray, list, dict, or scalar
Index	Unique hashable labels; default RangeIndex(n)
Dtype	Forces storage type over inference
Copy	copy=False shares the input buffer

Vectorized Operations

Arithmetic runs as index-aligned ufuncs in compiled C over contiguous memory — s1 + s2 aligns labels, then loops once at NumPy speed instead of per-element in Python.

```
# standard python loops
data = [1, 2, 3, 4, 5]
squares = []
for x in data:
    squares.append(x * x)
```

```
# vectorized
import numpy as np

data = np.array([1, 2, 3, 4, 5])
squares = data ** 2
```

Why Use a Series

Arithmetic runs as index-aligned ufuncs in compiled C over contiguous memory — s1 + s2 aligns labels, then loops once at NumPy speed instead of per-element in Python.

```
# manual alignment by hand
q1 = {"a": 10, "b": 20}
q2 = {"b": 5, "c": 7}
out = {k: q1.get(k,0) + q2.get(k,0)
      for k in q1.keys() | q2.keys()}
```

```
# Series: aligned + vectorized
s1 = pd.Series({"a": 10, "b": 20})
s2 = pd.Series({"b": 5, "c": 7})
s1.add(s2, fill_value=0)
# a 10.0  b 25.0  c 7.0
```

Missing Data Semantics

- np.nan is float-only, so an int64 Series upcasts to float64 on the first missing value; nullable Int64/boolean/string dtypes use pd.NA with a separate mask instead.

- NA propagates through element-wise ops but is skipped by reductions, and pd.NA follows three-valued logic in boolean comparisons.

- Advantage: missingness is first-class — .isna(), .fillna(method/value), .dropna(), .interpolate() handle gaps in place, with no sentinel values or manual masking.

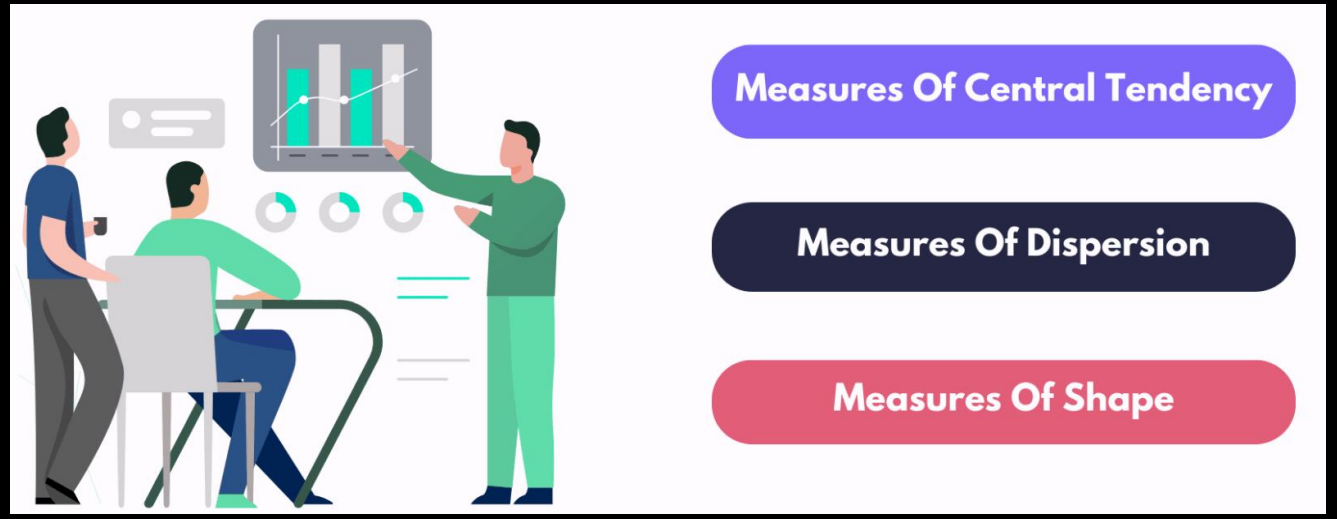
	GOV	CAPITAL	POPULATION	language
A	Iraq	Baghdad	100	NaN
B	Pakistan	Islamabad	300	NaN
C	UAE	DUBAI	120	NaN
D	UK	LONDON	150	NaN

Descriptive Statistics

Reductions (.mean(), .std(ddof=1), .median(), .quantile(), .describe()) run as single C passes over the values buffer and act only on numeric dtypes.

.idxmax()/.idxmin() return index labels rather than positions, and .value_counts(), .rank(), .cumsum() cover frequency and running-window work without loops.

- Advantage: NA is skipped by default (skipna=False to opt out), so summary statistics are correct on ragged real-world data with no pre-cleaning step.

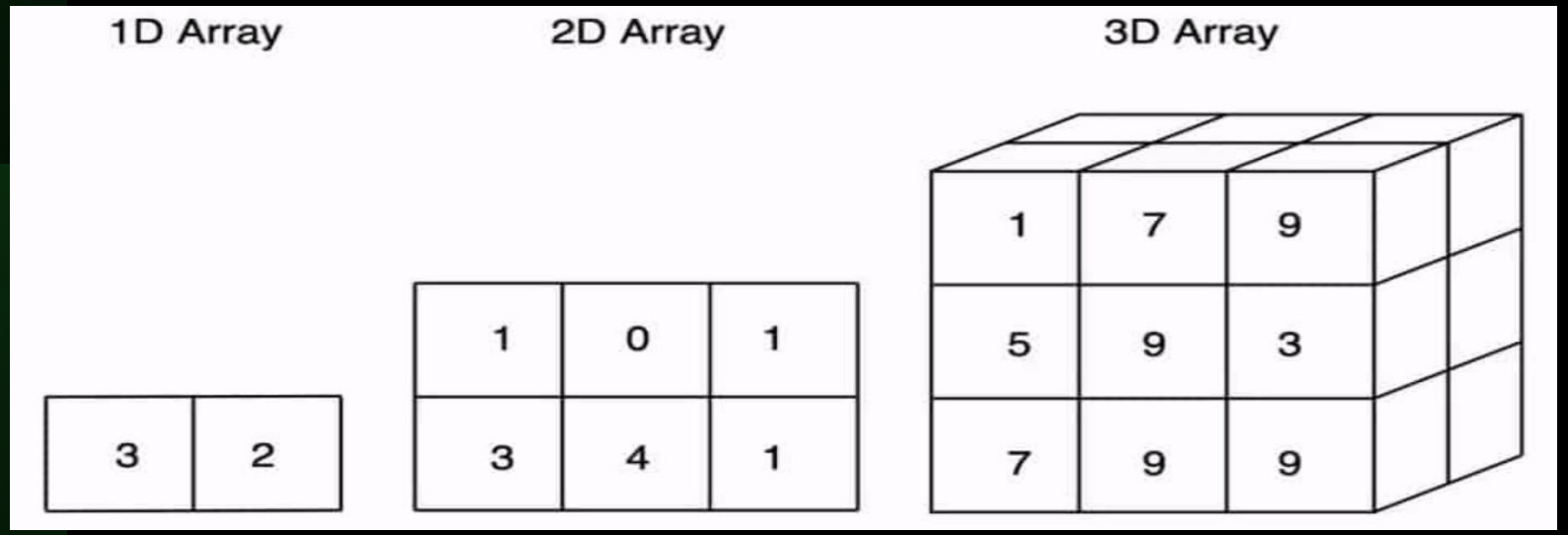


Performance & Memory Model

Backed by a single contiguous typed buffer: int64/float64 cost 8 bytes per element with no per-object overhead, so scans are cache-friendly and SIMD-friendly.

Object dtype stores 8-byte pointers to boxed Python objects — typically an order of magnitude slower and several times larger; category dtype often cuts memory dramatically for repeated strings.

Inspect real cost with .memory_usage(deep=True); use .to_numpy(copy=False) rather than .values for a stable, dtype-aware view of the buffer.



Interoperability & Conversion

Zero-friction conversion in both directions: pd.Series(list/dict/ndarray), .to_list(), .to_dict(), .to_numpy(), .to_frame(), plus .astype() for explicit dtype changes.

Advantage: a DataFrame column is a Series, so every Series method composes with groupby, merge, resample and the Arrow/Parquet I/O layer without conversion cost.

